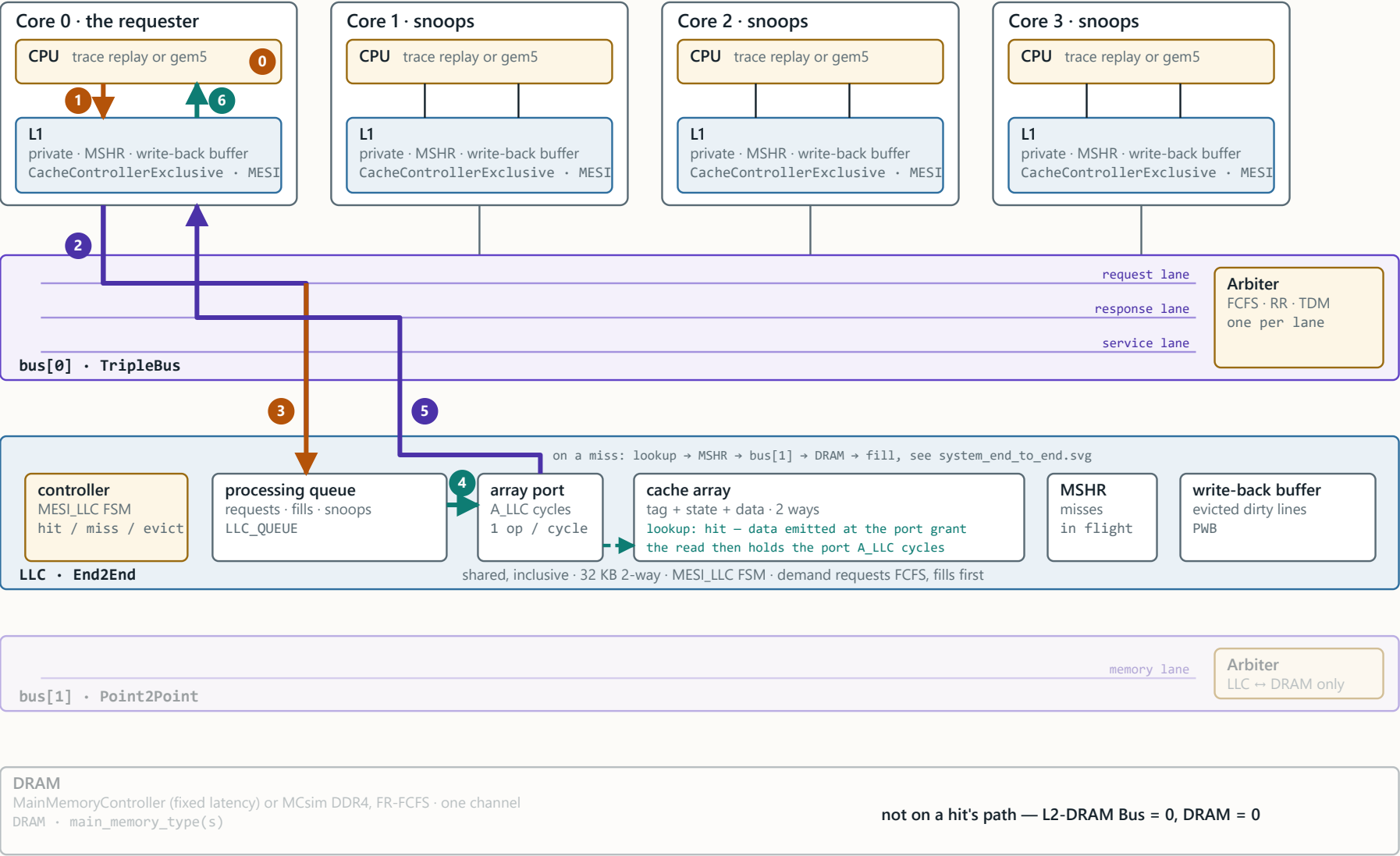


The system, end to end — one LLC hit from core 0, and the column that measures each hop

MultiCoreSystem as shipped: 4 cores, snoop MESI on a TripleBus, one shared LLC, a point-to-point memory bus, DRAM. Colour = the LatencyReport column; ①–⑥ are the columns a hit touches, in report order — L2-DRAM Bus and DRAM are 0.



The route, hop by hop

stamp → stamp is the column's span

- ① Ready; waits for a slot in the OoO window
Ready Cycle → CPU.ENTER
CPU Latency
- ② Issued; queued at L1
CPU.ENTER → L1.ENTER
L1 Stall
- ③ L1 misses, MSHR; wins the request lane
L1.ENTER → REQ_BUS.EXIT
Request Bus
- ④ Waits in the LLC queue; every L1 snooped it
REQ_BUS.EXIT → LLC.ENTER
L2 Stall
- ⑤ Waits for the port; hit, data out at the grant
LLC.ENTER → LLC.SERVICE = EXIT
L2 Access
- ⑥ Response lane back to core 0
LLC.EXIT → RESP_BUS.EXIT
Response Bus
- ⑦ L1 fills, CPU retires it
RESP_BUS.EXIT → CPU.EXIT
L1 Access

L2-DRAM Bus, DRAM 0 — the request never leaves the LLC.

Total = delivered – issued = ① ... ⑥, tiled exactly; ① lies before it. ④ ends at the port grant, not after A_LLC cycles: the data leaves at the grant, and the read's port time is paid by whoever waits for the port next (their ④), or a fill's write). Nothing under ③ ④ is a DRAM or memory-bus effect: on a hit, a slow request is the bus, the LLC queue, or the port. An L1 hit is shorter still: only ① ② ⑥ exist, the bus is idle.

- The other cores their L1s see every request on the request lane; an owner answers on the response lane in the LLC's place, its stamps standing in for the LLC's in ③ ④.
- The service lane the LLC's back-invalidations on an eviction and their echoes; not on this request's path.
- Directory protocol the same boxes: requests become point-to-point and the LLC-side directory forwards them instead of the L1s snooping.

waiting: in a queue, for a lane, for the port moving: a bus transfer being served: an array or DRAM access off the critical path